Iterative Improvement: Maximum Flow

Design and Analysis of Algorithms

Dhannya SM

Dept of CSE
SSN College of Engineering

Flow Networks

- A flow network is a directed graph used to model the transportation (or flow) of items.
- The transportation network can be represented by a connected weighted directed graph with n vertices numbered from 1 to n and a set of edges E, with the following properties:
  a) It contains exactly one vertex with no entering edges known as the source (generally numbered 1).
  b) It contains exactly one vertex with no leaving edges known as the sink, and assumed to be numbered n.
  c) The weight u_{ij} of each directed edge (i, j) is a positive integer, called the edge capacity. This number represents the upper bound on the amount of the material that can be sent from i to j through a link represented by this edge.

Flow Networks

- All vertices except the source and sink can serve only as redirection points without consuming or adding any amount of the material.
- In other words, the total amount of the material entering an intermediate vertex must be equal to the total amount of the material leaving the vertex. This condition is called the *flow-conservation* requirement.
- If we denote the amount sent through edge (i, j) by x_{ij} ,then for any intermediate vertex i,the flow-conservation requirement can be expressed by the following equality constraint:

$$\sum_{j:(j,i)\in E} x_{ji} = \sum_{j:(i,j)\in E} x_{ij}$$

where the sums in the left- and right-hand sides express the total inflow and outflow entering and leaving vertex i, respectively.

Flow Networks

- Since no amount of the material can be lost or gained by going through intermediate vertices of the network, it follows that the total amount of material leaving the source must end up at the sink.

$$\sum_{j:(1,j)\in E} x_{1j} = \sum_{j:(j,n)\in E} x_{jn}$$

- This quantity, the total outflow from the source or, equivalently, the total inflow into the sink is called the *value* of the flow. We denote it by v.
- It is this quantity that we will want to maximize over all possible flows in a network.
- Thus, a (feasible) flow is an assignment of real numbers x_{ij} to edges (i, j) of a given network that satisfy flow-conservation constraints and the capacity constraints $0 \leq x_{ij} \leq u_{ij}$ for every edge $(i, j) \in E$.

Maximum Flow problem

- Problem statement: The maximum-flow problem can be stated formally as the following optimization problem:

$$\begin{aligned} \text{maximize} \quad & \sum_{j:(1,j)\in E} x_{1j} \\ \text{subject to} \quad & \sum_{j:(j,i)\in E} x_{ji} - \sum_{j:(i,j)\in E} x_{ij} = 0 \text{ for } i = 2, 3, \ldots, n-1 \\ & 0 \le x_{ij} \le u_{ij} \text{ for every edge } (i,j) \in E \end{aligned}$$

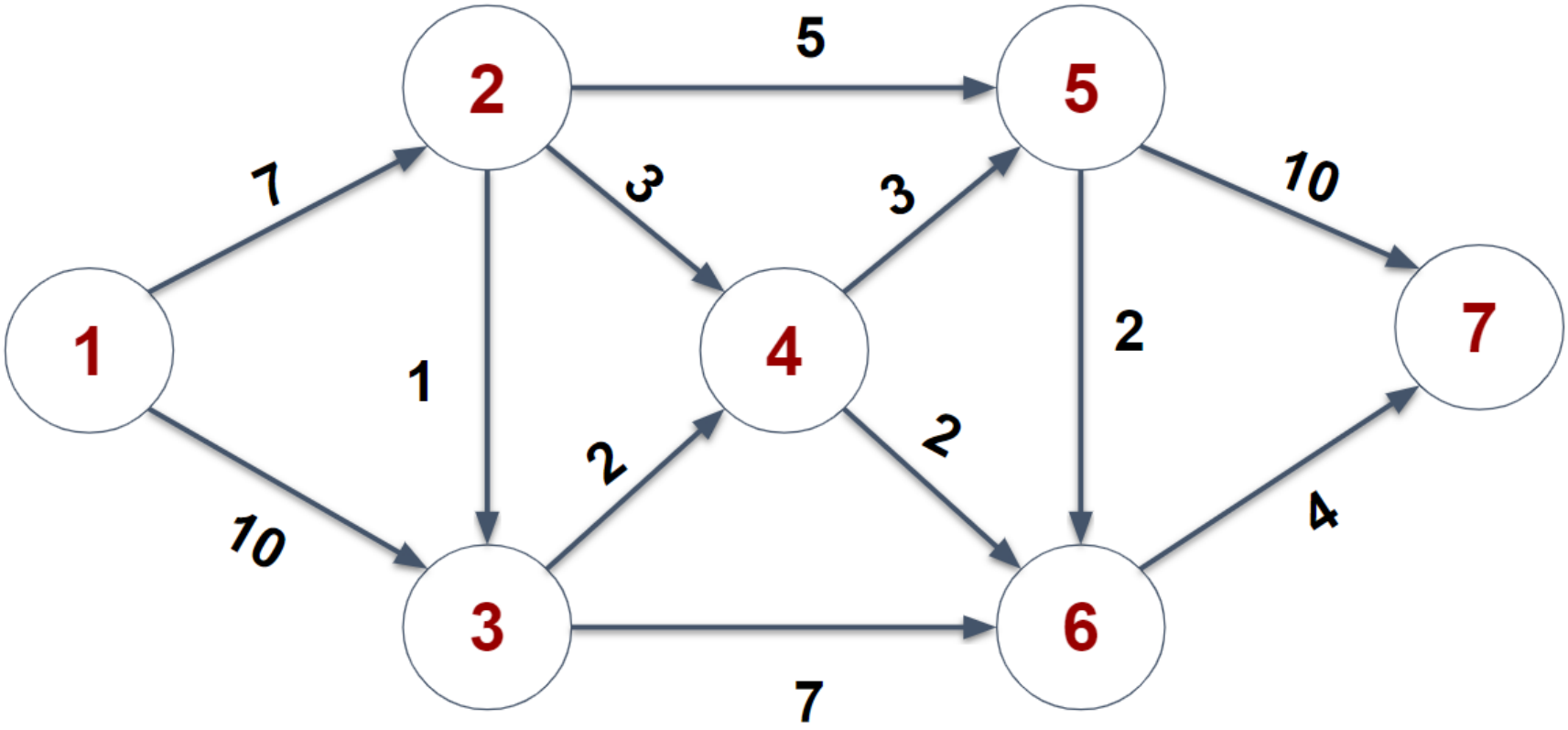
Iterative improvement idea

- We start with the zero flow (i.e., set $x_{ij} = 0$ for every edge (i, j) in the network).
- Then, on each iteration, we try to find a path from source to sink along which some additional flow can be sent.
- Such a path is called a *flow augmenting* (or an *augmenting*) path.
- If a flow-augmenting path is found, we adjust the flow along the edges of this path to get a flow of an increased value and try to find an augmenting path for the new flow.
- If no flow-augmenting path can be found, we conclude that the current flow is optimal.
- This general template for solving the maximum-flow problem is called the augmenting-path method, also known as the **Ford-Fulkerson method** after L. R. Ford, Jr., and D. R. Fulkerson, who discovered it.

Ford-Fulkerson method

- The Ford-Fulkerson method depends on three important ideas: **residual networks**, **augmenting paths**, and **cuts**.
- It starts with $x_{ij} = 0$ for every edge (i, j), giving an initial flow value 0, and iteratively increases the value of the flow.
- Each iteration increases the flow value in G by finding an augmenting path in an associated residual network G_f.
- Given a flow network G and a flow f, the residual network G_f consists of edges whose capacities represent how the flow can change on edges of G.
- An augmenting path is a path in the residual graph with unused capacity greater than zero from the source to the sink.

Example

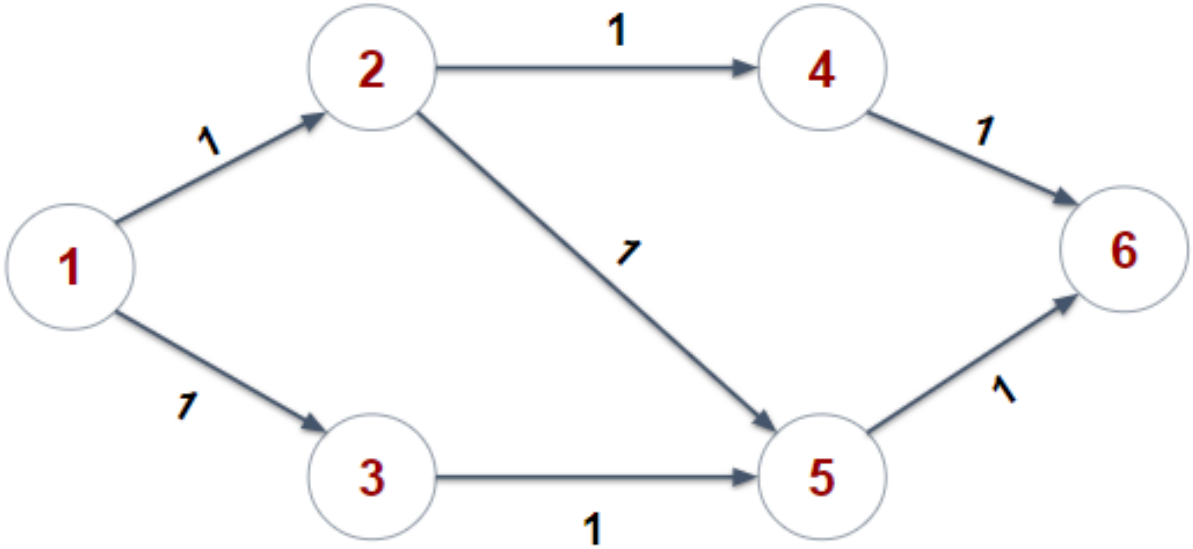


Algorithm

```
FORD-FULKERSON-METHOD(G, s, t)
initialize flow f to 0
while there exists an augmenting path p in the residual network G_f
    augment flow f along p
return f
```

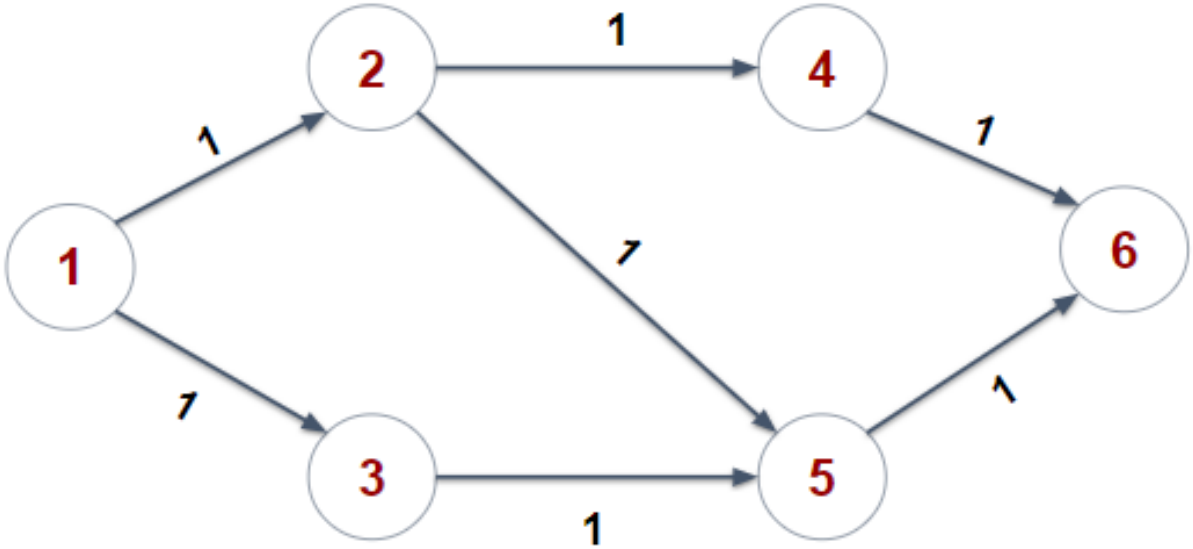
Example1

Consider the following flow network:



Example1

Consider the following flow network:



With the same technique as we used earlier, this network may not give max flow.

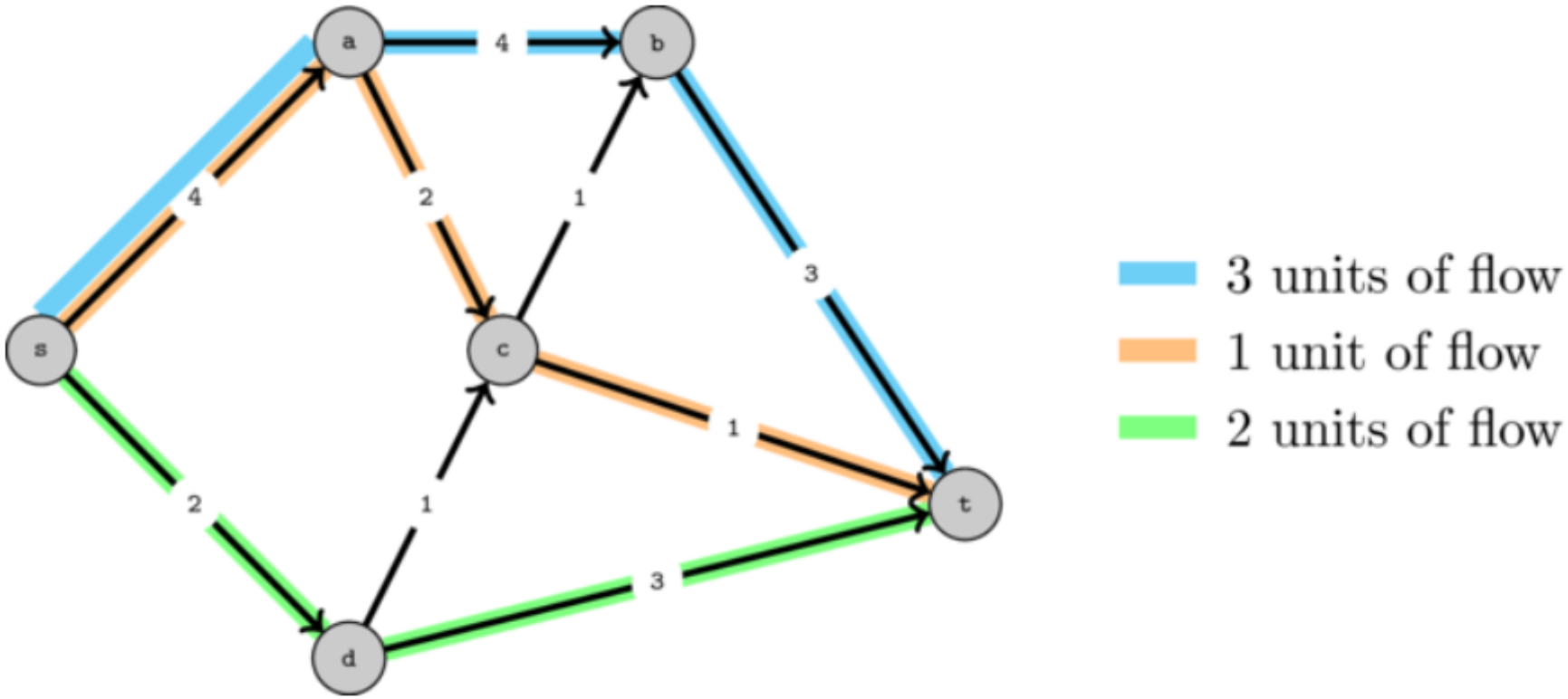
1 - 2 - 5 - 6

vs

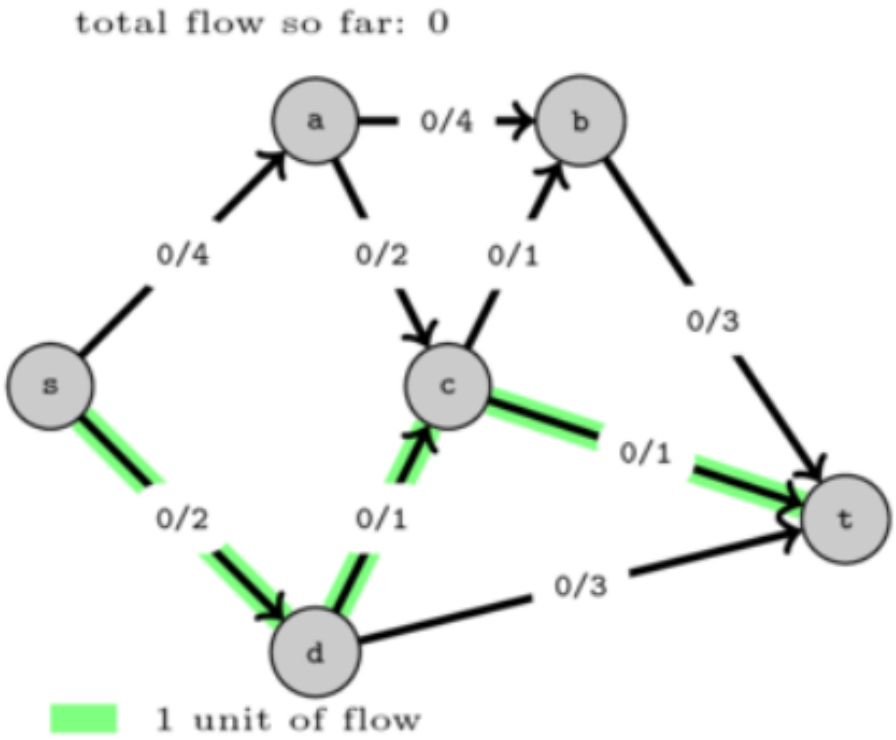
1 - 2 - 4 - 6 and 1 - 3 - 5 - 6

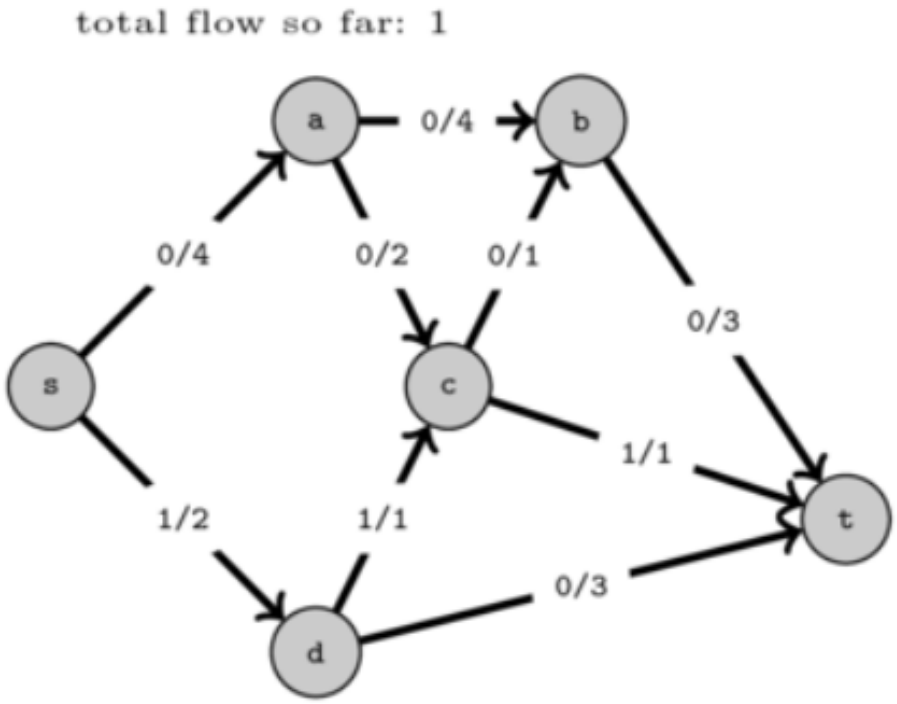
Example2

The following example and explanation are taken from:
https://inginious.org/course/competitive-programming/graphs-maxflow

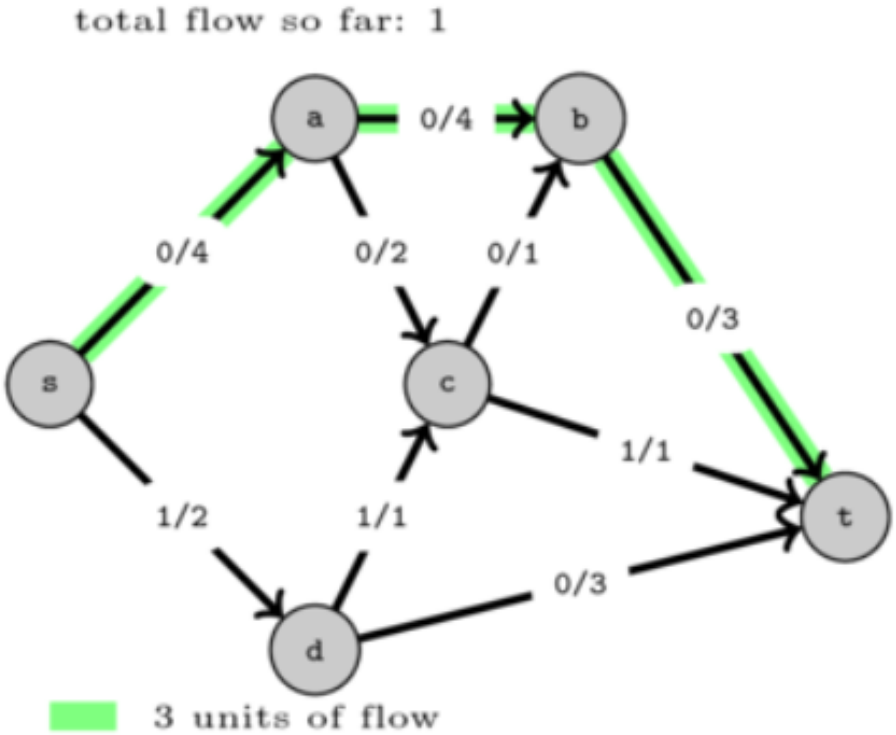


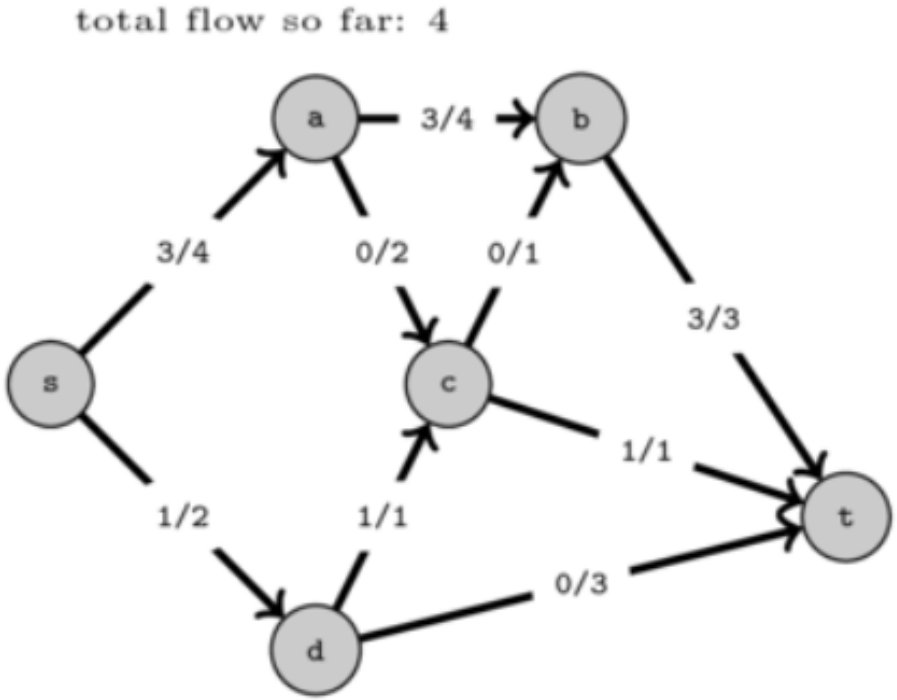
Example2 - alternate path



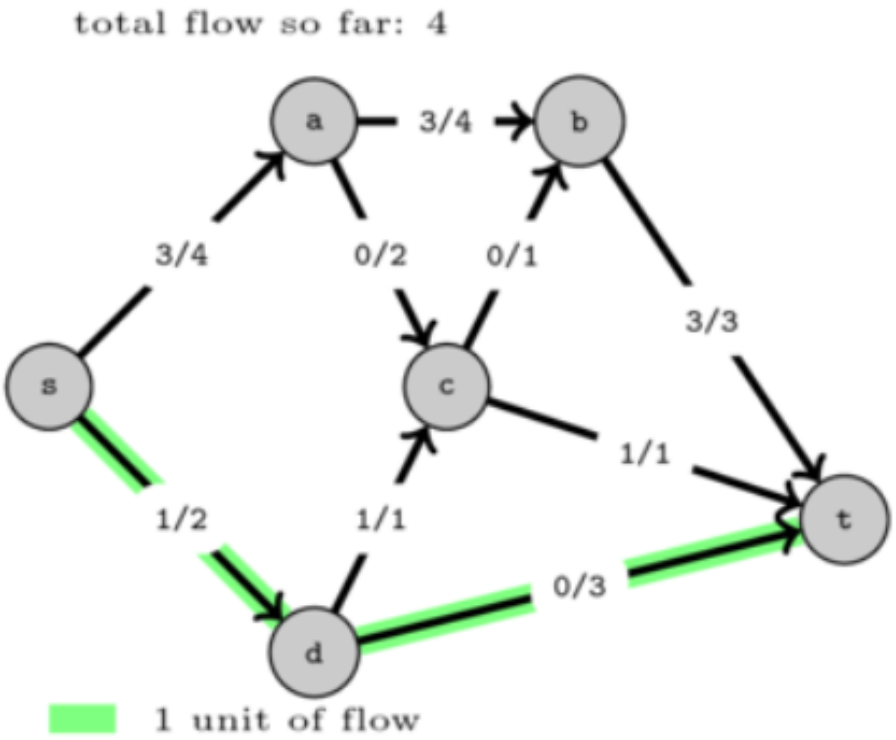


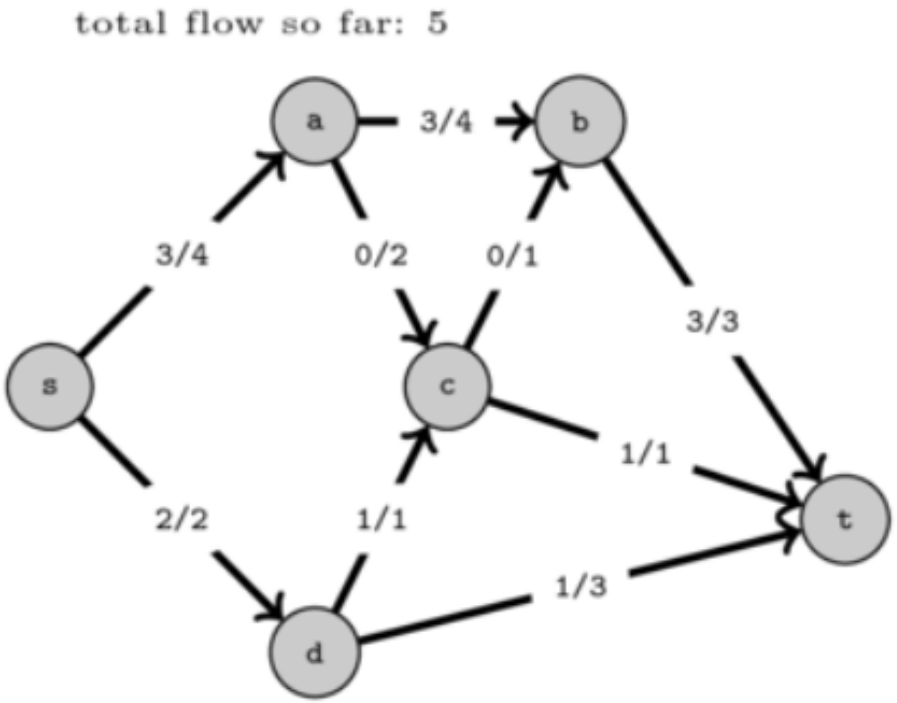
Example2


total flow so far: 1
a
b
s
c
t
d
0/4
0/4
0/2
0/1
0/3
1/1
1/2
1/1
0/3
3 units of flow

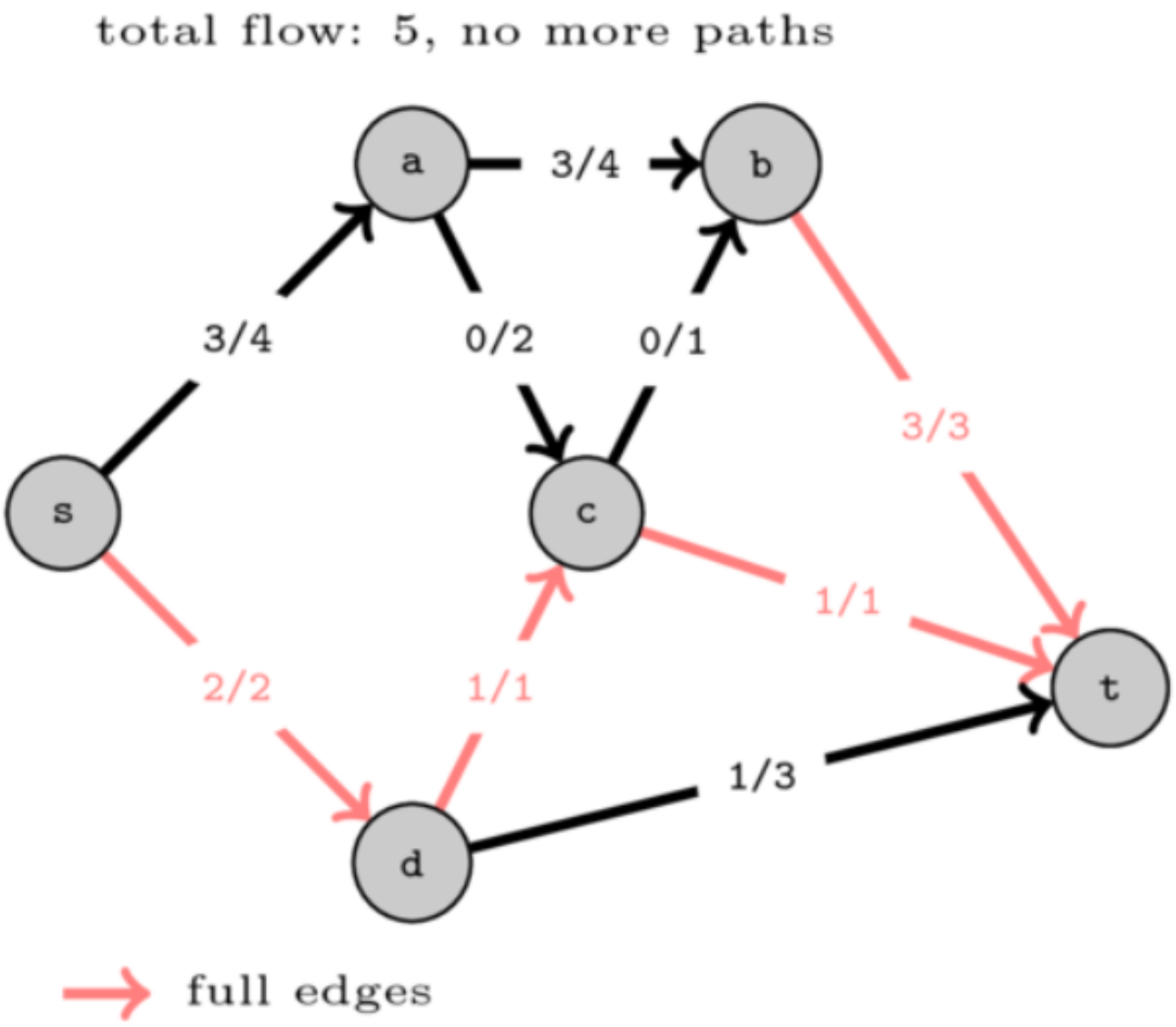

total flow so far: 4
a
b
s
c
t
d
3/4
3/4
0/2
0/1
3/3
1/1
1/2
1/1
0/3

Example2

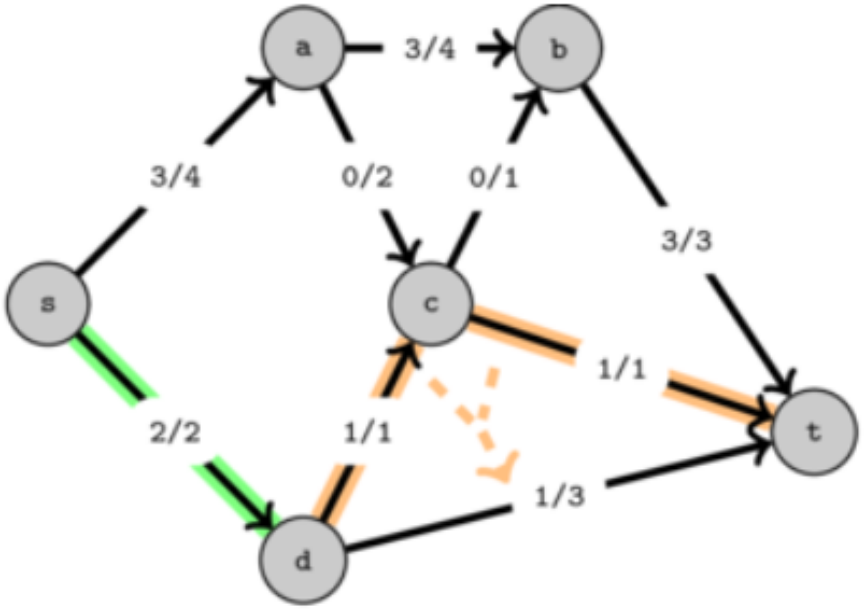

total flow so far: 4
a
b
s
c
t
d
3/4
3/4
0/2
0/1
3/3
1/1
1/2
1/1
0/3
1 unit of flow


total flow so far: 5
a
b
s
c
t
d
3/4
3/4
0/2
0/1
3/3
1/1
2/2
1/1
1/3

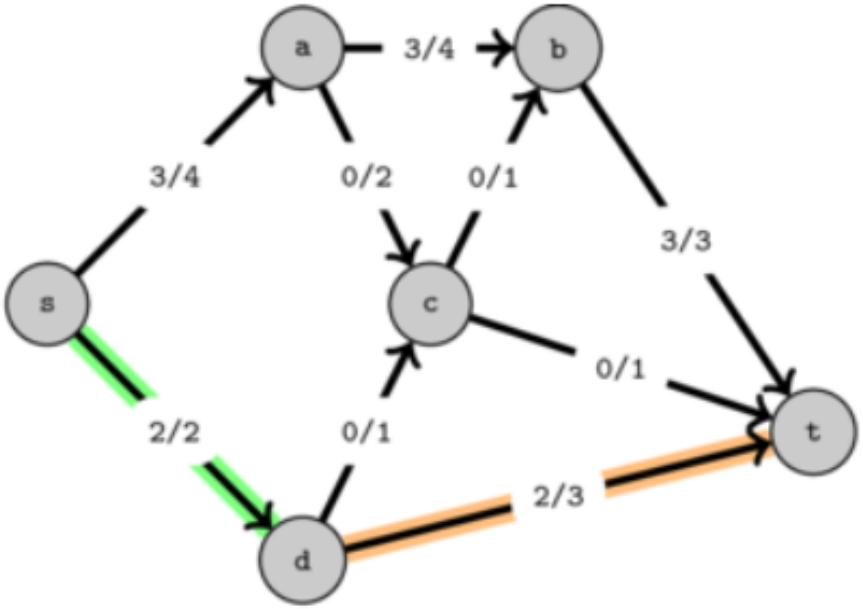
Example2


total flow: 5, no more paths
a
b
s
c
t
d
3/4
3/4
0/2
0/1
3/3
1/1
2/2
1/1
1/3
full edges

Example2

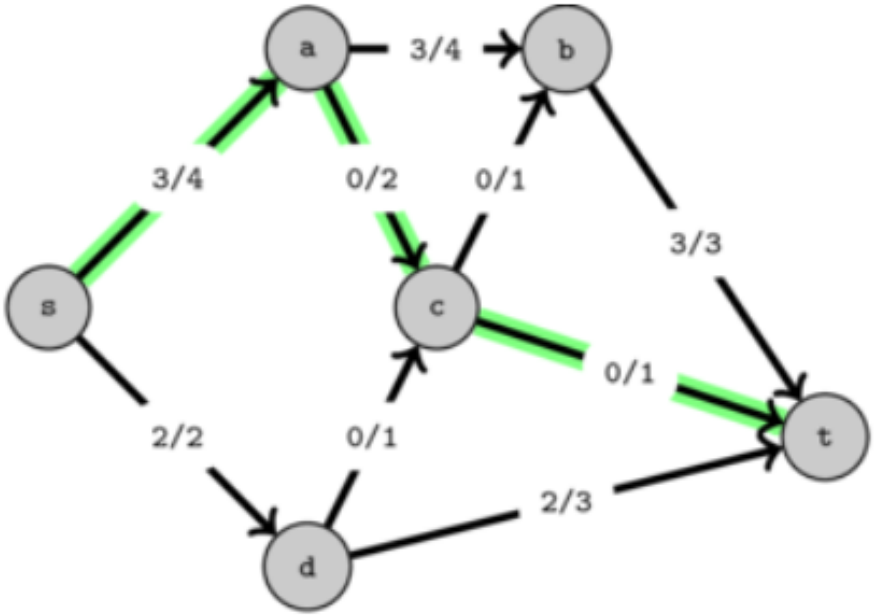


take 1 unit of flow from (d, c, t)

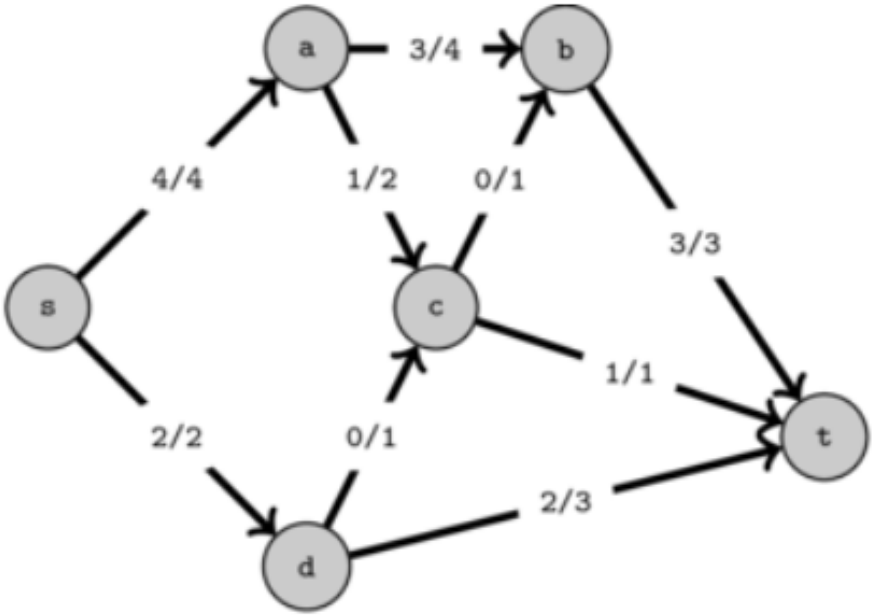


redirect it to (d, t)

Reverse Flow

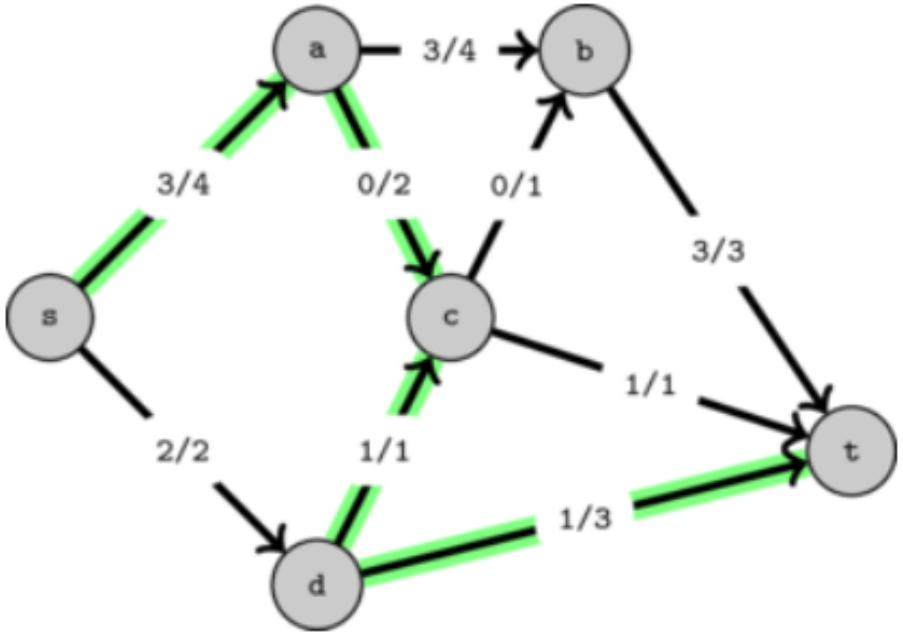


push a new unit on (s, a, c, t)



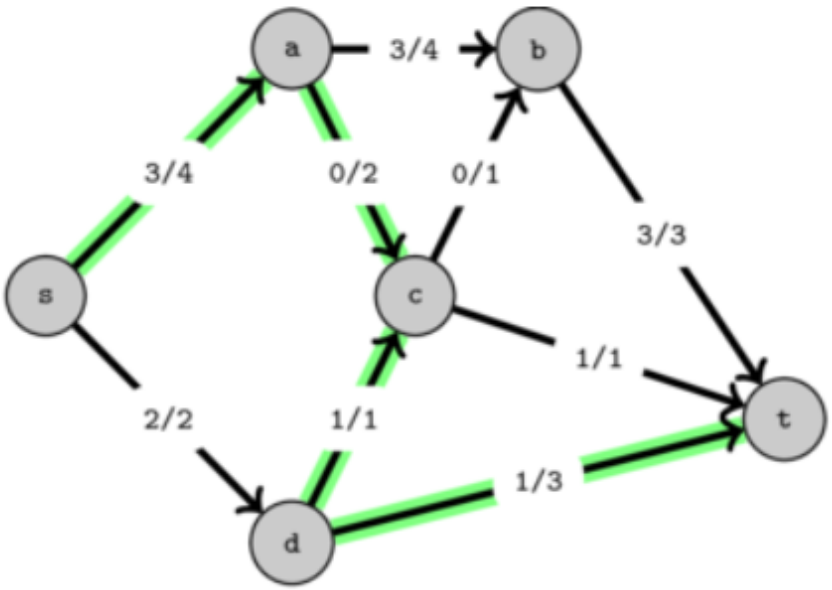
total flow $= 6$

Reverse Flow

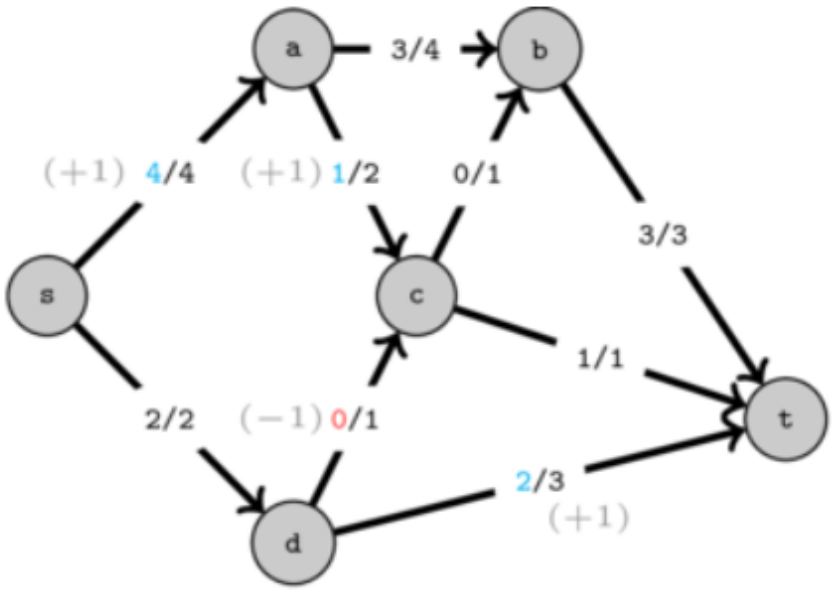


The two operations that we performed - the redirection plus pushing a new unit of flow - is the same as if we had pushed one unit of flow on the above path.

Reverse Flow



before pushing



after pushing

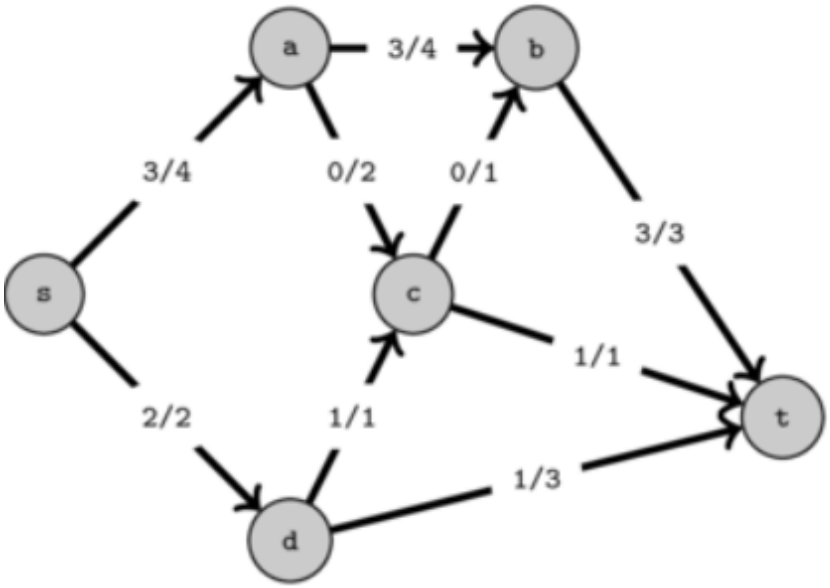
Reverse Flow

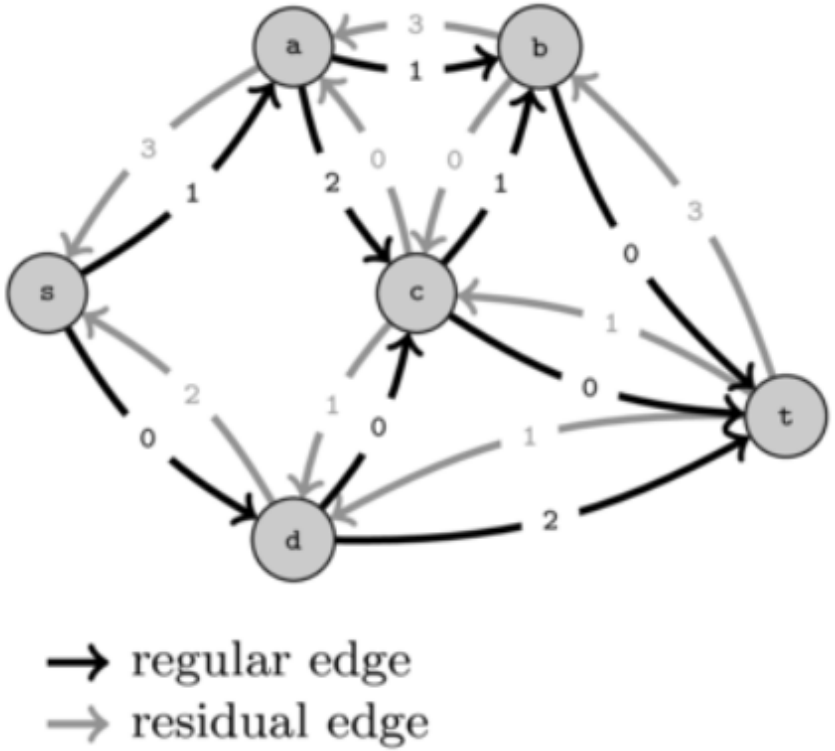
- The intuition behind this is that the part of the path until we cross an edge in the reverse direction corresponds to the new path on which we will push flow and the part of the path after the edge in reverse direction corresponds to where we redirect the flow.
- For every edge in the input graph, we will create a residual edge with the reverse direction initially with capacity 0.
- Then whenever we push flow f on one edge e, we decrease the capacity of e by f and increase the capacity of residual edge of e by f.

- This will allow us to find paths that traverse edges in reverse direction allowing us to redirect the flow passing on it towards another path. This is the residual graph.

Reverse Flow

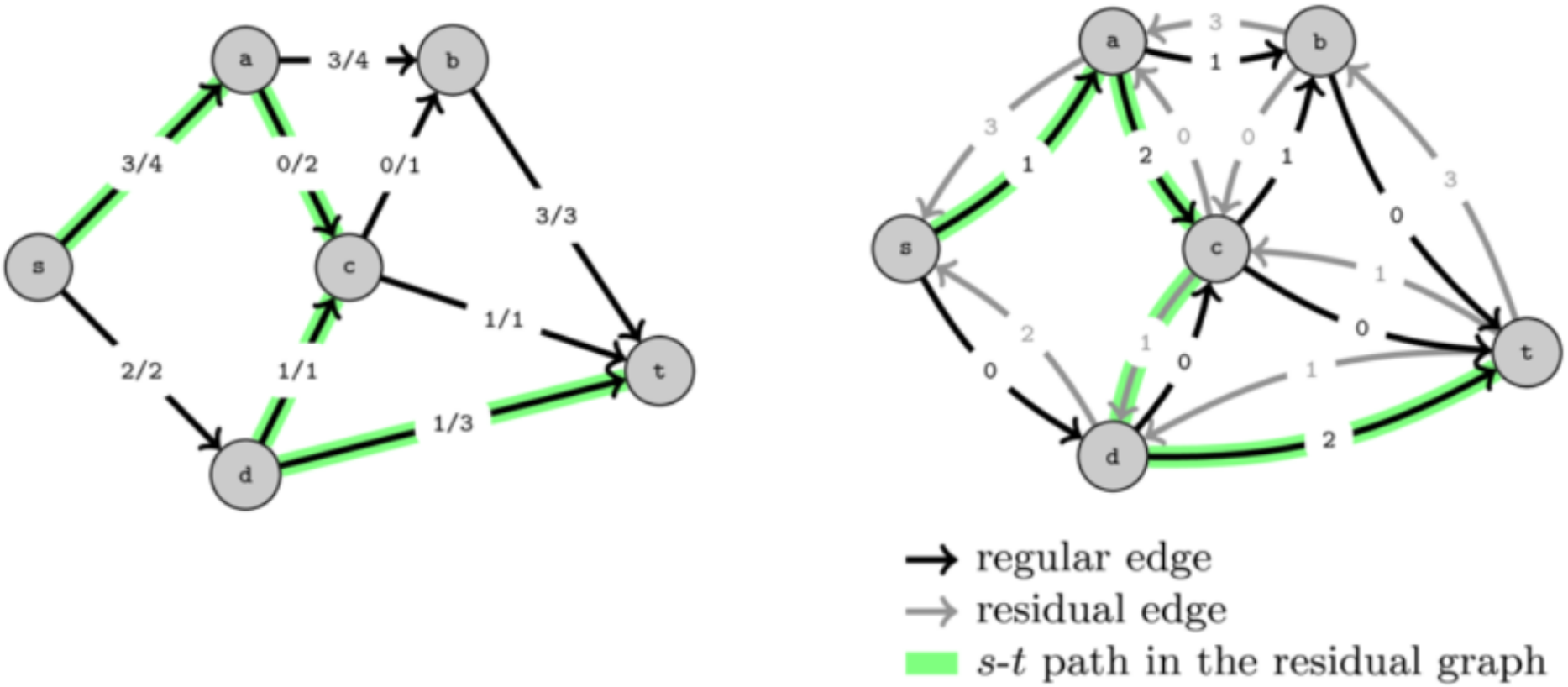
The figure below shows the residual network corresponding to the earlier example where we were only able to push 5 units of flow.





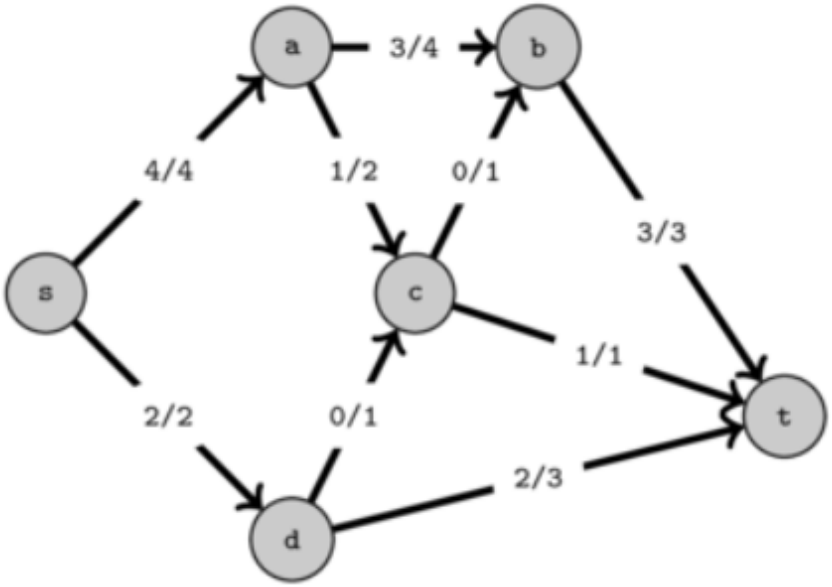
Reverse Flow

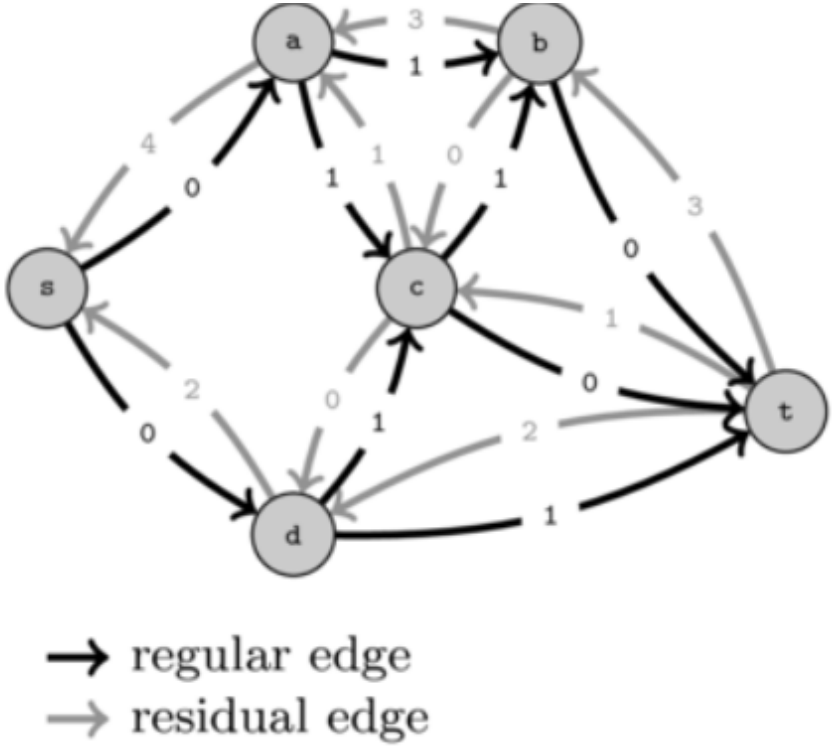
Using the residual graph, we can now very easily find new ways to push flow, we simply have to find a path of positive capacity from s to t. The following figure shows that the strange "path" that we used before, now corresponds to a normal path in the residual graph.



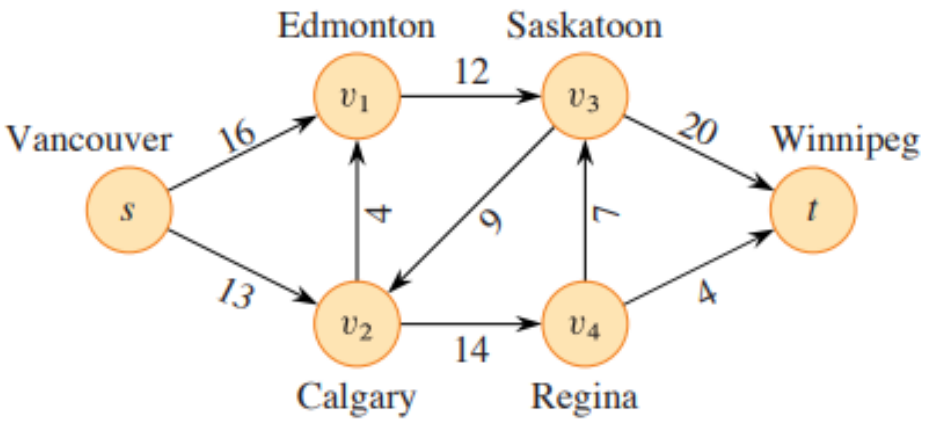
Reverse Flow

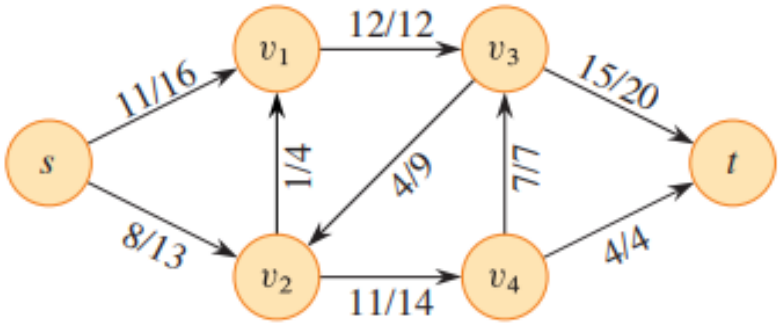
If we push one unit of flow on this path we indeed obtain the same flow as before and now we get a residual graph that contains no paths from s to t of positive capacity.



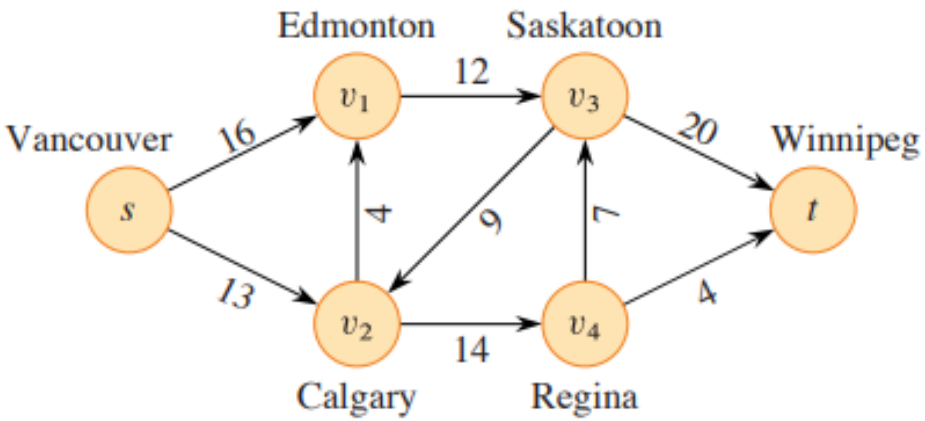


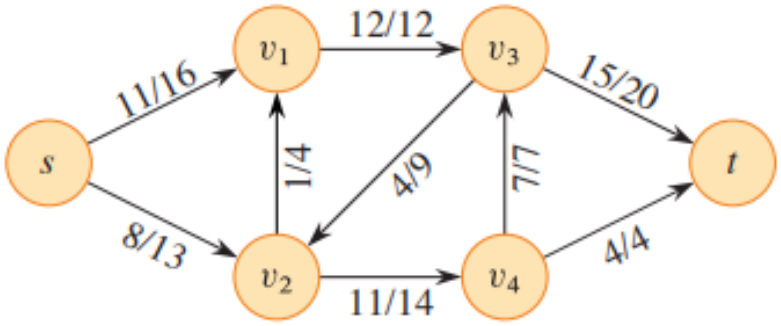
Example3


Edmonton
Saskatoon
Vancouver
Winnipeg
Calgary
Regina
s
t
v_1
v_2
v_3
v_4
16
13
12
4
9
7
14
20
4

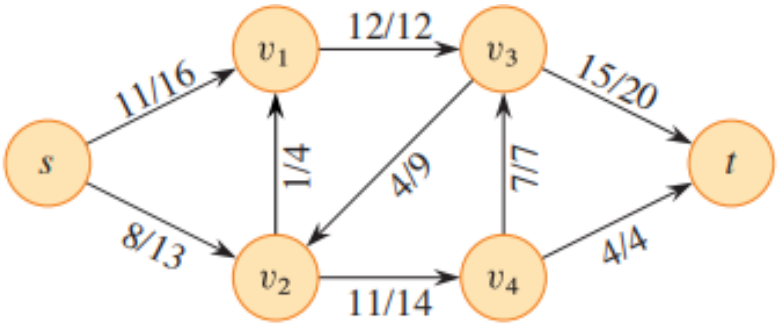

s
t
v_1
v_2
v_3
v_4
11/16
8/13
12/12
1/4
4/9
7/7
11/14
15/20
4/4

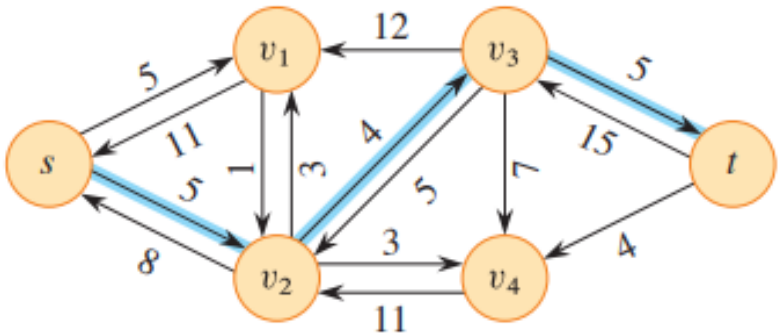
Example3



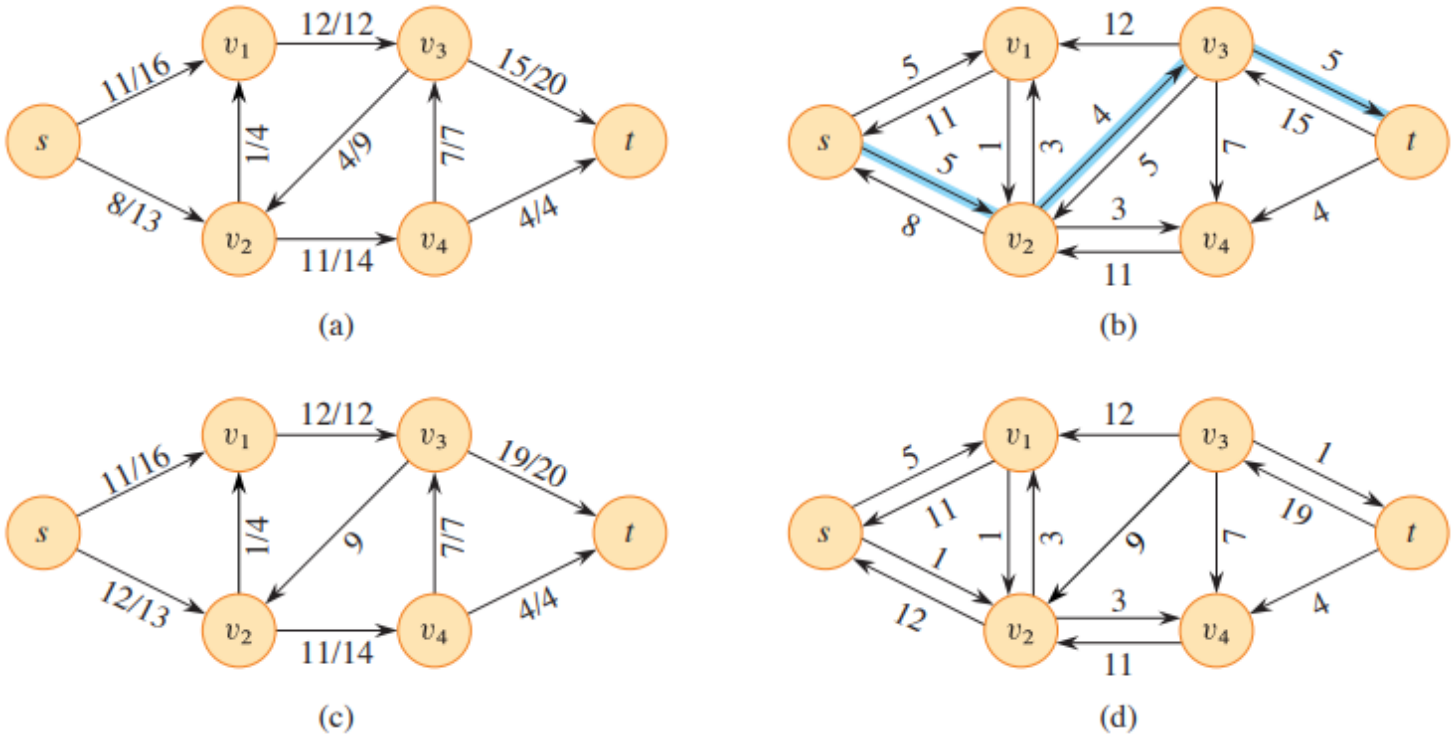


Residual graph:



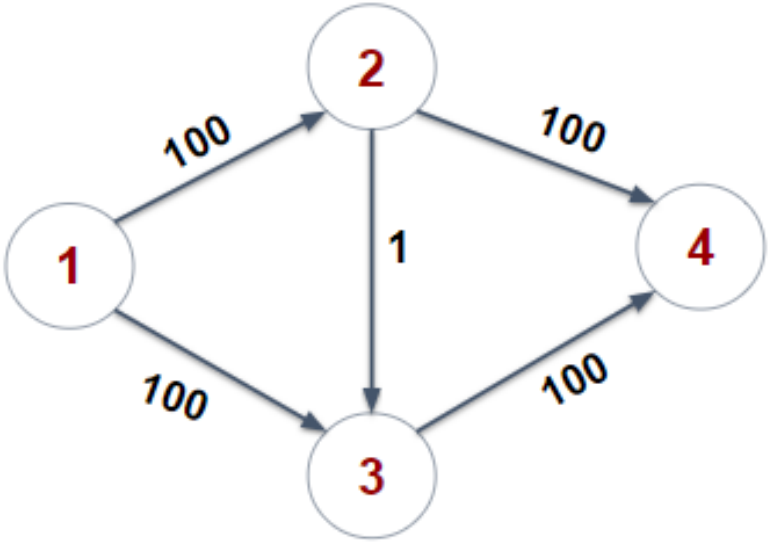


Reverse Flow



Runtime of FF Algorithm

Time complexity is $O(|E| * f)$, where f is the value of maximum flow.



Summary

- The maximum flow problem finds the value of the maximum flow in a flow network.
- The Ford-Fulkerson method uses the augmenting paths to produce residual networks, which in turn give rise to newer augmenting paths.
- The Max-Flow Min-Cut theorem: The value of a maximum flow in a network is equal to the capacity of its minimum cut.